

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM	:	123240003 123240042
NAMA	:	Bylbiss El Haqqie Shientya Belva
PLUG	:	IF-I
NAMA ASISTEN	:	Khatama Putra Hawla Khufyatul Qolbu

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Bylbiss El Haqqie
Shientya Belva

123240003
123240042

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung
Keputusan

Pada Tanggal:

Asisten Praktikum

Khatama Putra
NIM. 123230053

Asisten Praktikum

Hawla Khufyatul Qolbu
NIM. 123230021

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul Pemilihan Jam Tangan. Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 27 Mei 2026

Penyusun 1,

Penyusun 2,

Shientya Belva

123240042

Bylbiss El Haqqie

123240003

DAFTAR ISI

HALAMAN PENGESAHAN.....	ii
KATA PENGANTAR.....	iii
DAFTAR ISI.....	iv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang Masalah.....	1
1.2 Tujuan Proyek Akhir.....	1
1.3 Manfaat Proyek Akhir.....	1
BAB II PEMBAHASAN.....	2
2.1 Dasar Teori.....	2
2.2 Skenario Kasus & Dataset.....	2
2.3 Pemodelan Sistem Pendukung Keputusan.....	2
2.4 Perhitungan & Algoritma.....	2
2.5 Implementasi & Tampilan Sistem.....	2
BAB III JADWAL Pengerjaan DAN PEMBAGIAN TUGAS.....	3
3.1 Jadwal Pengerjaan.....	3
3.2 Pembagian Tugas.....	3
BAB IV KESIMPULAN DAN SARAN.....	4
4.2 Kesimpulan.....	4
4.3 Saran.....	4
DAFTAR PUSTAKA.....	5

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Pada perkembangan *smart devices* ini telah memunculkan beragam merek dan model *smartwatch* dengan spesifikasi yang bervariasi. Konsumen dihadapkan pada ratusan pilihan yang menawarkan keunggulan berbeda, seperti ukuran layar, ketahanan air, daya tahan baterai, dan harga yang tidak selalu sebanding dengan fitur. Kondisi ini sering menimbulkan kebingungan bagi para pembeli, karena tidak ada acuan yang objektif untuk membandingkan semua alternatif secara sekaligus. Akibatnya, keputusan pembelian cenderung didasarkan pada faktor subjektif atau promosi sesaat, bukan pada evaluasi menyeluruh terhadap kebutuhan dan preferensi pribadi. Ketiadaan alat bantu yang sistematis dalam proses seleksi *smartwatch* menjadi kendala utama yang perlu diatasi.

Untuk menjawab permasalahan tersebut, diperlukan sebuah sistem pendukung keputusan (SPK) yang dapat membantu pengguna menilai dan memeringkat *smartwatch* berdasarkan kriteria-kriteria penting. Penggunaan metode *Simple Additive Weighting* (SAW) karena kesederhanaan dan kemampuannya dalam menangani multikriteria dengan tipe benefit maupun cost. Dengan metode ini, setiap alternatif dinormalisasi terlebih dahulu, kemudian dikalikan dengan bobot preferensi pengguna, lalu dijumlahkan untuk memperoleh skor akhir. Hasil perangkingan yang transparan dan terstruktur akan memberikan rekomendasi yang objektif dan sesuai dengan kebutuhan masing-masing individu.

Oleh karena itu, proyek akhir ini mengembangkan aplikasi berbasis Streamlit bernama SmartWatchRank. Aplikasi ini tidak hanya menampilkan dataset *smartwatch* secara interaktif, tetapi juga menyediakan fitur penyesuaian bobot kriteria, filter merek, dan visualisasi grafis untuk memperjelas hasil perhitungan. Dengan adanya aplikasi ini, diharapkan pengguna dapat dengan mudah membandingkan berbagai *smartwatch*, serta mengambil keputusan pembelian yang lebih cerdas dan rasional.

1.2 Tujuan Proyek Akhir

- 1) Membangun sistem pendukung keputusan berbasis *website* untuk membantu pengguna memilih *smartwatch* terbaik sesuai preferensi pribadi.
- 2) Menerapkan metode *Simple Additive Weighting* (SAW) dalam proses perangkingan alternatif berdasarkan empat kriteria yaitu ukuran layar, ketahanan air, daya tahan baterai, dan harga.
- 3) Menyediakan antarmuka yang interaktif melalui *sidebar* dan *tabs* agar pengguna dapat mengatur bobot kriteria, memfilter merek, serta melihat hasil perhitungan secara langsung.
- 4) Menampilkan visualisasi data berupa diagram lingkaran, batang, dan garis untuk memudahkan analisis distribusi merek dan tren skor terhadap harga.

- 5) Menyimpan riwayat perhitungan sehingga pengguna dapat membandingkan hasil keputusan dari berbagai skenario bobot yang berbeda.

1.3 Manfaat Proyek Akhir

Proyek akhir SmartWatch yang dikembangkan menggunakan metode Simple Additive Weighting (SAW) memberikan berbagai manfaat bagi pengguna maupun dalam penerapan Sistem Pendukung Keputusan (SPK). Berdasarkan implementasi pada kode Python kami itu menggunakan framework Streamlit, berikut manfaat yang diperoleh:

1. Membantu Pengambilan Keputusan Secara Objektif
Aplikasi menghitung rekomendasi smartwatch berdasarkan data numerik dan bobot kriteria yang diberikan pengguna. Dengan metode SAW, proses pemilihan tidak hanya berdasarkan opini atau tren, tetapi berdasarkan perhitungan yang terukur dan transparan.
2. Pengguna Dapat Menyesuaikan Prioritas Kriteria
Sistem menyediakan pengaturan bobot untuk:
 - a. Ukuran layar
 - b. Ketahanan air
 - c. Daya tahan baterai
 - d. Harga
3. Hal ini memungkinkan setiap pengguna memperoleh rekomendasi sesuai kebutuhan pribadi. Misalnya, pengguna yang lebih mementingkan baterai dapat memberikan bobot lebih tinggi pada kriteria tersebut.
4. Mempermudah Perbandingan Banyak Smartwatch
Data smartwatch ditampilkan dalam bentuk tabel lengkap, ranking, dan visualisasi grafik sehingga pengguna dapat membandingkan banyak produk sekaligus tanpa harus menghitung secara manual. Sistem juga menampilkan ranking otomatis berdasarkan nilai SAW tertinggi.
5. Menyediakan Visualisasi Data yang Informatif
Aplikasi menampilkan:
 - a. Pie chart distribusi brand
 - b. Bar chart peringkat smartwatch
 - c. Line chart hubungan harga dengan SAW Score
6. Visualisasi ini membantu pengguna memahami data dengan lebih cepat dan intuitif
7. Menyimpan Riwayat Perhitungan
Sistem memiliki fitur penyimpanan riwayat hasil perhitungan ke file CSV sehingga pengguna dapat:
 - a. Melihat kembali hasil sebelumnya
 - b. Membandingkan beberapa konfigurasi bobot
 - c. Menyimpan hasil rekomendasi tanpa perlu menghitung ulang
8. Efisien dan Mudah Digunakan
Dengan antarmuka berbasis web menggunakan Streamlit, aplikasi dapat digunakan

secara interaktif dan user-friendly. Pengguna cukup mengatur slider bobot lalu menekan tombol “Hitung SAW”.

9. Meningkatkan Pemahaman Metode SAW

Aplikasi tidak hanya memberikan hasil akhir, tetapi juga menampilkan:

- a. Detail normalisasi
- b. Bobot akhir
- c. Nilai SAW setiap alternatif

10. Ini membuat sistem cocok digunakan sebagai media pembelajaran metode SAW dalam mata kuliah Sistem Pendukung Keputusan.

11. Mengurangi Risiko Salah Memilih Produk

Banyak pengguna bingung memilih smartwatch karena spesifikasi yang beragam. Sistem ini membantu menyederhanakan proses seleksi sehingga pengguna dapat memilih produk terbaik sesuai preferensi dan budget.

Singkatnya, proyek ini bukan sekadar ranking smartwatch, tapi mesin kecil untuk mengubah kebingungan jadi keputusan yang rasional. Kayak punya asisten belanja yang doyan matematika.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Simple Additive Weighting (SAW) merupakan salah satu metode dalam Sistem Pendukung Keputusan (SPK) yang digunakan untuk menentukan alternatif terbaik berdasarkan beberapa kriteria yang telah ditentukan. Metode ini sering disebut sebagai metode penjumlahan terbobot karena proses perhitungannya dilakukan dengan menjumlahkan nilai setiap kriteria yang telah dikalikan dengan bobot preferensinya masing-masing.

Metode SAW bekerja dengan cara melakukan normalisasi terhadap data agar seluruh kriteria memiliki skala yang sama. Setelah dinormalisasi, setiap nilai kriteria dikalikan dengan bobot yang mencerminkan tingkat kepentingan kriteria tersebut. Hasil perkalian kemudian dijumlahkan untuk memperoleh nilai akhir atau skor preferensi dari setiap alternatif. Alternatif dengan nilai tertinggi akan menjadi pilihan terbaik.

Pada aplikasi SmartWatch, metode SAW digunakan untuk menentukan smartwatch terbaik berdasarkan empat kriteria, yaitu:

1. Ukuran Layar (Display Size) – Kriteria Benefit
Semakin besar ukuran layar, semakin baik nilai yang diperoleh.
2. Ketahanan Air (Water Resistance) – Kriteria Benefit
Semakin tinggi tingkat ketahanan air, semakin baik nilainya.
3. Daya Tahan Baterai (Battery Life) – Kriteria Benefit
Semakin lama daya tahan baterai, semakin baik nilainya.
4. Harga (Price) – Kriteria Cost
Semakin rendah harga smartwatch, semakin baik nilainya.

Normalisasi Data: sebelum dilakukan perhitungan skor akhir, setiap nilai kriteria dinormalisasi agar dapat dibandingkan secara adil.

Untuk kriteria Benefit, digunakan rumus:

$$r_{ij} = \frac{x_{ij}}{\max(x_j)}$$

Keterangan:

1. r_{ij} = nilai normalisasi
2. x_{ij} = nilai alternatif ke-i pada kriteria ke-j
3. $\max(x_j)$ = nilai maksimum pada kriteria ke-j

Untuk kriteria Cost, digunakan rumus:

$$r_{ij} = \frac{\min(x_j)}{x_{ij}}$$

Keterangan:

1. $\min(x_j)$ = nilai minimum pada kriteria ke-j

Perhitungan Nilai Preferensi: Setelah normalisasi selesai dilakukan, nilai preferensi setiap alternatif dihitung menggunakan rumus:

$$V_i = \sum_{j=1}^n w_j \times r_{ij}$$

Keterangan:

1. V_i = nilai preferensi alternatif ke-i
2. w_j = bobot kriteria ke-j
3. r_{ij} = nilai normalisasi alternatif ke-i pada kriteria ke-j
4. n = jumlah kriteria

Alternatif dengan nilai V_i terbesar akan memperoleh peringkat tertinggi dan direkomendasikan sebagai pilihan terbaik.

Dalam aplikasi SmartWatch, metode SAW memungkinkan pengguna memperoleh rekomendasi smartwatch terbaik secara objektif berdasarkan kombinasi spesifikasi produk dan tingkat kepentingan setiap kriteria yang ditentukan pengguna.

2.2 Skenario Kasus & Dataset

A. Skenario Kasus

Perkembangan teknologi wearable device menyebabkan semakin banyak pilihan smartwatch yang tersedia di pasaran. Setiap produk menawarkan spesifikasi yang berbeda-beda, seperti ukuran layar, ketahanan air, daya tahan baterai, serta harga. Banyaknya alternatif yang tersedia sering kali membuat calon pembeli mengalami kesulitan dalam menentukan smartwatch yang paling sesuai dengan kebutuhan dan preferensinya.

Dalam kondisi tersebut, diperlukan suatu Sistem Pendukung Keputusan (SPK) yang mampu membantu proses pemilihan smartwatch secara objektif berdasarkan beberapa

kriteria yang dianggap penting oleh pengguna. Oleh karena itu, dikembangkan aplikasi SmartWatch yang menerapkan metode Simple Additive Weighting (SAW) untuk melakukan proses evaluasi dan perankingan berbagai alternatif smartwatch.

Pengambil keputusan: dalam sistem ini adalah calon pembeli atau pengguna smartwatch yang ingin mencari produk terbaik sesuai preferensi masing-masing.

Tujuan utama dari sistem adalah:

1. Menentukan smartwatch terbaik dari sejumlah alternatif yang tersedia.
2. Membantu pengguna membandingkan berbagai produk secara objektif.
3. Menghasilkan rekomendasi smartwatch berdasarkan bobot kepentingan yang ditentukan pengguna.
4. Menyajikan hasil perankingan yang transparan dan mudah dipahami.

Kriteria yang digunakan: berdasarkan implementasi pada aplikasi, terdapat empat kriteria utama yang digunakan dalam proses pengambilan keputusan:

1. Ukuran Layar (Display Size) – Kriteria Benefit
Semakin besar ukuran layar, semakin baik nilai yang diperoleh.
2. Ketahanan Air (Water Resistance) – Kriteria Benefit
Semakin tinggi tingkat ketahanan air, semakin baik nilainya.
3. Daya Tahan Baterai (Battery Life) – Kriteria Benefit
Semakin lama daya tahan baterai, semakin baik nilainya.
4. Harga (Price) – Kriteria Cost
Semakin rendah harga smartwatch, semakin baik nilainya.

B. Dataset

Dataset yang digunakan dalam proyek ini merupakan dataset smartwatch yang diperoleh dari platform Kaggle.

Dataset berisi informasi spesifikasi berbagai smartwatch dari berbagai merek terkenal seperti Apple, Samsung, Garmin, Fitbit, Huawei, Amazfit, Fossil, dan merek lainnya. Berdasarkan hasil pengolahan data pada aplikasi, dataset terdiri dari:

1. 379 data smartwatch
2. 42 brand smartwatch
3. Berbagai spesifikasi teknis yang dapat digunakan sebagai dasar pengambilan keputusan

Atribut dataset: dataset asli memiliki beberapa atribut, namun aplikasi SmartWatch menggunakan atribut berikut sebagai dasar perhitungan SAW:

1. Brand, Merek smartwatch

2. Model, Nama model smartwatch
3. Display Size (inches), Ukuran layar dalam inci
4. Resolution, Resolusi layar
5. Water Resistance (meters), Tingkat ketahanan air (meter)
6. Battery Life (days), Lama daya tahan baterai (hari)
7. Price (USD), Harga smartwatch dalam Dollar Amerika

Sumber Dataset: Kaggle – Smartwatch Price Dataset Using Clustering

Dataset ini dipilih karena:

1. Memiliki jumlah data yang cukup besar sehingga dapat menghasilkan proses perankingan yang lebih representatif.
2. Memuat spesifikasi penting yang relevan dengan kebutuhan pengguna smartwatch.
3. Memiliki atribut yang sesuai dengan metode SAW yang membutuhkan kriteria bertipe benefit dan cost.
4. Data berasal dari berbagai merek sehingga hasil rekomendasi menjadi lebih objektif dan tidak terfokus pada satu brand tertentu.

2.3 Pemodelan Sistem Pendukung Keputusan

Pada proyek akhir ini, Sistem Pendukung Keputusan (SPK) digunakan untuk menentukan smartwatch terbaik dengan menerapkan metode Simple Additive Weighting (SAW). Sistem melakukan evaluasi terhadap setiap alternatif berdasarkan beberapa kriteria yang telah ditentukan, kemudian menghasilkan peringkat berdasarkan nilai preferensi tertinggi.

A. Alternatif

Alternatif merupakan pilihan smartwatch yang akan dibandingkan. Untuk mempermudah pemodelan, digunakan contoh data sederhana sebagai berikut:

Kode	Alternatif
A1	Apple Watch Series 9
A2	Samsung Galaxy Watch 6
A3	Garmin Forerunner 255
A4	Huawei Watch GT 4
A5	Amazfit GTR 4

B. Kriteria

Kriteria yang digunakan mengacu pada implementasi aplikasi SmartWatchRank.

Kode	Kriteria	Jenis
C1	Ukuran Layar (Display Size)	Benefit
C2	Ketahanan Air (Water Resistance)	Benefit
C3	Daya Tahan Baterai (Battery Life)	Benefit
C4	Harga (Price)	Cost

Keterangan:

1. **Benefit** → semakin besar nilainya semakin baik.
2. **Cost** → semakin kecil nilainya semakin baik.

C. Skala Penilaian

Agar proses perhitungan lebih mudah dilakukan, setiap nilai spesifikasi dikonversi ke dalam skala penilaian 1–5.

Ukuran Layar	Nilai
≤ 1.2 inci	1
$> 1.2 - 1.4$ inci	2
$> 1.4 - 1.6$ inci	3
$> 1.6 - 1.8$ inci	4
> 1.8 inci	5

Ketahanan Air	Nilai
≤ 10 m	1
$> 10 - 30$ m	2
$> 30 - 50$ m	3
$> 50 - 100$ m	4
> 100 m	5

Daya Tahan Baterai	Nilai
≤ 2 hari	1
$> 2 - 5$ hari	2
$> 5 - 10$ hari	3
$> 10 - 20$ hari	4
> 20 hari	5

Harga (USD)	Nilai
> 500	1
401 – 500	2
301 – 400	3
201 – 300	4
≤ 200	5

Karena harga merupakan kriteria cost, maka smartwatch dengan harga lebih murah memperoleh nilai yang lebih tinggi.

D. Matriks Keputusan

Contoh hasil konversi alternatif ke dalam skala penilaian:

Alternatif	C1	C2	C3	C4
A1	4	4	2	2
A2	4	4	2	3
A3	3	5	5	3
A4	4	4	4	4
A5	4	4	5	4

E. Bobot Kriteria

Contoh bobot yang digunakan dalam perhitungan SAW:

Kriteria	Bobot
C1 – Ukuran Layar	0,25
C2 – Ketahanan Air	0,25
C3 – Daya Tahan Baterai	0,25
C4 – Harga	0,25

Total bobot = 1,00.

Bobot ini bersifat dinamis karena nantinya pengguna dapat mengubah tingkat kepentingan masing-masing kriteria melalui slider yang tersedia pada sidebar aplikasi. Setelah itu bobot akan dinormalisasi secara otomatis sehingga total bobot selalu bernilai 1.00.

2.4 Perhitungan & Algoritma

Langkah-Langkah Algoritma SAW

1. Menentukan Alternatif

Langkah pertama adalah menentukan alternatif yang akan dievaluasi. Dalam penelitian ini, alternatif berupa berbagai produk smartwatch yang terdapat pada dataset.

Contoh alternatif:

1. A1 = Apple Watch Series 9
2. A2 = Samsung Galaxy Watch 6
3. A3 = Garmin Forerunner 255
4. A4 = Huawei Watch GT 4
5. A5 = Amazfit GTR 4

2. Menentukan Kriteria

Setiap alternatif dinilai berdasarkan empat kriteria yang digunakan dalam aplikasi.

Kode	Kriteria	Jenis
C1	Ukuran Layar	Benefit
C2	Ketahanan Air	Benefit
C3	Daya Tahan Baterai	Benefit
C4	Harga	Cost

Kriteria benefit berarti semakin besar nilainya semakin baik, sedangkan kriteria cost berarti semakin kecil nilainya semakin baik.

3. Menentukan Bobot Kriteria

Setiap kriteria diberikan bobot sesuai tingkat kepentingannya.

Contoh:

Kriteria	Bobot
C1	0,25
C2	0,25
C3	0,25
C4	0,25

Pada aplikasi SmartWatch, bobot dapat diatur langsung oleh pengguna melalui slider dan akan dinormalisasi secara otomatis sehingga total bobot sama dengan 1.

4. Membentuk Matriks Keputusan

Data smartwatch disusun ke dalam bentuk matriks keputusan yang berisi nilai setiap alternatif pada masing-masing kriteria.

Contoh:

Alternatif	C1	C2	C3	C4
A1	1,8	50	3	450
A2	1,5	50	2	350
A3	1,3	100	14	300

5. Melakukan Normalisasi

Karena setiap kriteria memiliki satuan yang berbeda, maka diperlukan proses normalisasi.

Untuk kriteria Benefit:

$$R_{ij} = \frac{X_{ij}}{\max(X_j)}$$

Untuk kriteria Cost:

$$R_{ij} = \frac{\min(X_j)}{X_{ij}}$$

Keterangan:

1. R_{ij} = nilai normalisasi
2. X_{ij} = nilai alternatif pada kriteria tertentu
3. $\max(X_j)$ = nilai maksimum kriteria
4. $\min(X_j)$ = nilai minimum kriteria

Pada implementasi program:

1. Display Size → Benefit
2. Water Resistance → Benefit
3. Battery Life → Benefit
4. Price → Cost

6. Menghitung Nilai Preferensi

Setelah normalisasi selesai dilakukan, setiap nilai normalisasi dikalikan dengan bobot kriterianya masing-masing.

Rumus nilai preferensi:

$$V_i = \sum_{j=1}^n (W_j \times R_{ij})$$

Keterangan:

1. V_i = nilai akhir alternatif ke-i
2. W_j = bobot kriteria ke-j
3. R_{ij} = nilai normalisasi alternatif ke-i
4. n = jumlah kriteria

Nilai V_i merupakan skor akhir SAW yang digunakan untuk menentukan peringkat.

7. Melakukan Perangkingan

Semua alternatif diurutkan berdasarkan nilai preferensi (V_i) dari yang terbesar hingga terkecil.

1. Nilai tertinggi → Peringkat 1
2. Nilai terendah → Peringkat terakhir

Alternatif dengan skor SAW tertinggi dianggap sebagai smartwatch yang paling sesuai dengan preferensi pengguna.

2.5 Implementasi & Tampilan Sistem

```
import streamlit as st
import pandas as pd
import numpy as np
```

```

import matplotlib.pyplot as plt
from datetime import datetime
import warnings

warnings.filterwarnings("ignore")
import os
import json

# Konfigurasi Halaman
st.set_page_config(
    page_title="SmartWatchRank",
    page_icon="🕒",
    layout="wide",
    initial_sidebar_state="expanded",
)

# CSS Tampilan
st.markdown(
    """
<style>
    .stApp {
        background: linear-gradient(135deg, #f5e6d3 0%, #e8d5b7 100%);
    }
    [data-testid="stSidebar"] {
        background-color: #3e2723;
    }
    [data-testid="stSidebar"] * {
        color: #f5e6d3 !important;
    }
    [data-testid="stSidebar"] .stTextInput input {
        color: black !important;
        background-color: white !important;
        border-radius: 5px;
    }
    [data-testid="stSidebar"] .stMultiSelect [data-baseweb="select"]
input::placeholder {
        color: #4e342e !important;
        opacity: 1 !important;
    }
    [data-testid="stSidebar"] .stMultiSelect [data-baseweb="select"]
input {
        color: #4e342e !important;
    }
    [data-testid="stSidebar"] .stMultiSelect [data-baseweb="tag"] span {
        color: #4e342e !important;
    }
    [data-testid="stSidebar"] .stMultiSelect input {
        color: #4e342e !important;
    }
    [data-testid="stSidebar"] .stMultiSelect [data-baseweb="select"]
span[data-testid="stMarkdownContainer"] {
        color: #c0c0c0 !important;
    }
    [data-testid="stSidebar"] .stMultiSelect [data-baseweb="select"] {
        background-color: #f5f5f5 !important;
        border-radius: 5px;
    }
    [data-testid="stSidebar"] .stMultiSelect [data-baseweb="select"] div
{

```

```

        color: black !important;
    }
    [data-testid="stSidebar"] .stMultiSelect svg {
        fill: #c49a6c !important;
    }
    div[data-testid="stSidebar"] ul[role="listbox"] li {
        color: black !important;
        background-color: white !important;
    }
    .stButton > button {
        background-color: #8b5a2b !important;
        color: white !important;
        border-radius: 8px !important;
        width: 100%;
    }
    h1, h2, h3 {
        color: #4e342e !important;
    }
    .metric-card {
        background-color: #fff8f0;
        border-radius: 15px;
        padding: 15px;
        text-align: center;
        box-shadow: 0 4px 10px rgba(0,0,0,0.1);
        border-bottom: 3px solid #c49a6c;
    }
    .metric-value {
        font-size: 28px;
        font-weight: bold;
        color: #6d4c41;
    }
    .metric-label {
        font-size: 14px;
        color: #8d6e63;
    }
    .history-card {
        background-color: #fff8f0;
        border-radius: 10px;
        padding: 15px;
        margin-bottom: 10px;
        border-left: 4px solid #8b5a2b;
    }
    .stTabs {
        margin-top: 10px;
        margin-bottom: 20px;
    }
    .stTabs [data-baseweb="tab-list"] {
        display: flex;
        flex-wrap: wrap;
        gap: 8px;
        background-color: transparent;
        border-bottom: none;
        padding: 0;
    }
    .stTabs [data-baseweb="tab"] {
        background-color: #fff8f0;
        border-radius: 30px !important;
        padding: 8px 24px !important;
        font-size: 1rem;
    }

```

```

        font-weight: 500;
        color: #4e342e !important;
        border: 1px solid #c49a6c;
        transition: all 0.2s ease;
        white-space: nowrap;
    }
    .stTabs [data-baseweb="tab"]:hover {
        background-color: #e8d5b7;
        border-color: #8b5a2b;
    }
    .stTabs [aria-selected="true"] {
        background-color: #8b5a2b !important;
        color: white !important;
        border-color: #8b5a2b;
    }
    /* Hilangkan garis bawah default Streamlit */
    .stTabs [data-baseweb="tab-highlight"] {
        display: none;
    }
    div[data-testid="column"] {
        display: flex;
        flex-direction: column;
        justify-content: stretch;
    }
    .metric-card {
        height: 100%;
        display: flex;
        flex-direction: column;
        justify-content: center;
        transition: transform 0.1s ease;
    }
    .metric-card:hover {
        transform: translateY(-3px);
    }
    .metric-value {
        font-size: 32px; /* sedikit lebih besar */
    }
    .metric-label {
        font-size: 14px;
        font-weight: 500;
        color: #5d3a1a;
    }
}
</style>
"""
    unsafe_allow_html=True,
)

# Persistensi Data (Riwayat Perhitungan)
HISTORY_FILE = "history_data.csv"
RESULTS_DIR = "saved_results"

if not os.path.exists(RESULTS_DIR):
    os.makedirs(RESULTS_DIR)

def load_history_from_csv():
    # Membaca riwayat dari file CSV
    # Bobot disimpan sebagai JSON string, perlu di-parse
    if os.path.exists(HISTORY_FILE):

```

```

        df = pd.read_csv(HISTORY_FILE)
        # Konversi ke list of dict (tanpa kolom 'top_watch' yang mungkin
        berupa dict)
        history = []

        for _, row in df.iterrows():
            # weights disimpan sebagai string JSON, perlu di-parse
            weights = json.loads(row["weights_json"])
            entry = {
                "id": int(row["id"]),
                "timestamp": row["timestamp"],
                "name": row["name"],
                "weights": weights,
                "top_watch": {
                    "brand": row["top_brand"],
                    "model": row["top_model"],
                    "score": float(row["top_score"]),
                },
                "total_watches": int(row["total_watches"]),
            }
            history.append(entry)
        return history
    return []

def save_history_to_csv(history):
    # Menyimpan riwayat ke file CSV
    # Menjaga konsistensi data antar sesi
    if not history:
        return
    rows = []
    for h in history:
        rows.append(
            {
                "id": h["id"],
                "timestamp": h["timestamp"],
                "name": h["name"],
                "weights_json": json.dumps(
                    h["weights"]
                ), # simpan bobot sebagai JSON string
                "top_brand": h["top_watch"]["brand"],
                "top_model": h["top_watch"]["model"],
                "top_score": h["top_watch"]["score"],
                "total_watches": h["total_watches"],
            }
        )
    df = pd.DataFrame(rows)
    df.to_csv(HISTORY_FILE, index=False)

def load_saved_result(result_id):
    # Membaca DataFrame hasil perhitungan berdasarkan id
    # Setiap hasil disimpan sebagai file CSV terpisah
    file_path = os.path.join(RESULTS_DIR, f"result_{result_id}.csv")
    if os.path.exists(file_path):
        return pd.read_csv(file_path)
    return None

```

```

def save_saved_result(result_id, df_result):
    # Menyimpan DataFrame hasil perhitungan ke file CSV

    file_path = os.path.join(RESULTS_DIR, f"result_{result_id}.csv")
    df_result.to_csv(file_path, index=False)

# INISIALISASI SESSION STATE
# Session state menjaga data saat pengguna berinteraksi tanpa reload
# Riwayat dan hasil perhitungan dimuat dari file saat aplikasi dimulai

if "history" not in st.session_state:
    loaded_history = load_history_from_csv()
    st.session_state.history = loaded_history

if "saved_results" not in st.session_state:
    # Muat saved_results dari file berdasarkan id di history
    saved = {}
    for h in st.session_state.history:
        result_df = load_saved_result(h["id"])
        if result_df is not None:
            saved[h["id"]] = result_df
    st.session_state.saved_results = saved

if "saw_result" not in st.session_state:
    st.session_state.saw_result = None # Hasil SAW terakhir
if "norm_df" not in st.session_state:
    st.session_state.norm_df = None # Matriks normalisasi
if "norm_details" not in st.session_state:
    st.session_state.norm_details = None # Parameter normalisasi (min,
max)

# LOAD DATA
# Cache data agar tidak membaca file CSV setiap interaksi
@st.cache_data
def load_data():
    try:
        df = pd.read_csv("smart_watch.csv")
        cols_needed = [
            "Brand",
            "Model",
            "Display Size (inches)",
            "Resolution",
            "Water Resistance (meters)",
            "Battery Life (days)",
            "Price (USD)",
        ]
        available = [c for c in cols_needed if c in df.columns]
        if len(available) < 5:
            st.error(
                "Kolom tidak lengkap. Pastikan file CSV memiliki: Brand,
Model, Display Size (inches), Water Resistance (meters), Battery Life
(days), Price (USD) "
            )
        return pd.DataFrame()
        df = df[available].copy()
        df["Price (USD)"] = (
            df["Price (USD)"]

```

```

        .astype(str)
        .str.replace(r"[\$,]", "", regex=True)
        .astype(float)
    )
    for col in [
        "Display Size (inches)",
        "Water Resistance (meters)",
        "Battery Life (days)",
    ]:
        if col in df.columns:
            df[col] = pd.to_numeric(df[col], errors="coerce")
    df = df.dropna()
    return df
except Exception as e:
    st.error(f"Error loading CSV: {e}")
    return pd.DataFrame()

# FUNGSI SAW
# Kriteria benefit dinormalisasi dengan max, cost dengan min
def process_saw(df, weights):
    result = df.copy()
    norm_df = pd.DataFrame(index=df.index)
    norm_details = {}

    max_disp = df["Display Size (inches)"].max()
    norm_df["Display Size"] = df["Display Size (inches)"] / max_disp
    norm_details["Display Size"] = f"Benefit, max = {max_disp:.2f}"

    max_water = df["Water Resistance (meters)"].max()
    norm_df["Water Resistance"] = df["Water Resistance (meters)"] / max_water
    norm_details["Water Resistance"] = f"Benefit, max = {max_water:.0f}"

    max_bat = df["Battery Life (days)"].max()
    norm_df["Battery Life"] = df["Battery Life (days)"] / max_bat
    norm_details["Battery Life"] = f"Benefit, max = {max_bat:.1f}"

    min_price = df["Price (USD)"].min()
    norm_df["Price"] = min_price / df["Price (USD)"]
    norm_details["Price"] = f"Cost, min = ${min_price:.0f}"

    result["SAW_Score"] = (
        norm_df["Display Size"] * weights["Display Size"]
        + norm_df["Water Resistance"] * weights["Water Resistance"]
        + norm_df["Battery Life"] * weights["Battery Life"]
        + norm_df["Price"] * weights["Price"]
    )
    result["Rank"] = result["SAW_Score"].rank(ascending=False,
method="min").astype(int)
    result = result.sort_values("SAW_Score",
ascending=False).reset_index(drop=True)

    # Hasil diurutkan dari skor tertinggi ke terendah
    return result, norm_df, norm_details

# FUNGSI VISUALISASI
def plot_pie_gradient(data):

```

```

# Distribusi brand teratas
if data.empty:
    fig, ax = plt.subplots(figsize=(2.5, 2.5))
    ax.text(0.5, 0.5, "Data kosong", ha="center", va="center")
    return fig
brand_counts = data["Brand"].value_counts().head(5)
cmap = plt.cm.get_cmap("YlOrBr")
    colors = [cmap(i / len(brand_counts)) for i in
range(len(brand_counts))]
fig, ax = plt.subplots(figsize=(2.5, 2.5))
ax.pie(
    brand_counts.values,
    labels=brand_counts.index,
    autopct="%1.1f%%",
    colors=colors,
    textprops={"fontsize": 8, "color": "#4e342e"},
    wedgeprops={"edgecolor": "white", "linewidth": 0.8},
)
for autotext in ax.texts:
    if autotext.get_text().endswith("%"):
        autotext.set_color("white")
        autotext.set_fontweight("bold")
        autotext.set_fontsize(7)
n_brands = len(brand_counts)
ax.set_title(
    f"Distribusi {n_brands} Brand Terbanyak",
    fontsize=10,
    color="#4e342e",
    fontweight="bold",
)
fig.patch.set_facecolor("#f5e6d3")
return fig

def plot_bar_gradient(data):
    # Top smartwatch berdasarkan SAW score
    if data.empty:
        fig, ax = plt.subplots(figsize=(3.5, 2.5))
        ax.text(0.5, 0.5, "Data kosong", ha="center", va="center")
        return fig
    top10 = data.head(10).copy()
    top10["Label"] = top10["Brand"] + " " + top10["Model"]
    scores = top10["SAW_Score"]
    cmap = plt.cm.get_cmap("YlOrBr")
    colors = [cmap(i / len(scores)) for i in range(len(scores))]
    fig, ax = plt.subplots(figsize=(3.5, 2.5))
    bars = ax.barh(
        top10["Label"], scores, color=colors, edgecolor="#5d3a1a",
linewidth=0.8
    )
    ax.set_xlabel("SAW Score", fontsize=8, color="#4e342e")

    n_items = len(top10)
    ax.set_title(
        f"Top {n_items} Smartwatch Berdasarkan SAW Score",
        fontsize=10,
        color="#4e342e",
        fontweight="bold",
    )
)

```

```

ax.invert_yaxis()
ax.set_facecolor("#faf0e6")
fig.patch.set_facecolor("#f5e6d3")
for bar, val in zip(bars, scores):
    ax.text(
        bar.get_width() + 0.005,
        bar.get_y() + bar.get_height() / 2,
        f"{val:.3f}",
        va="center",
        ha="left",
        fontsize=7,
        color="#4e342e",
    )
plt.tight_layout()
return fig

def plot_line_chart_harga_saw(data):
    # Tren SAW Score terhadap harga
    if data.empty or len(data) < 2:
        fig, ax = plt.subplots(figsize=(5, 3.5))
        ax.text(0.5, 0.5, "Data tidak cukup untuk plot", ha="center",
va="center")
        fig.patch.set_facecolor("#f5e6d3")
        return fig
    sorted_data = data.sort_values("Price (USD)")
    x = sorted_data["Price (USD)"]
    y = sorted_data["SAW_Score"]
    fig, ax = plt.subplots(figsize=(5, 3.5))
    ax.plot(
        x,
        y,
        color="#8b5a2b",
        linewidth=1.5,
        marker="o",
        markersize=3,
        markerfacecolor="#c49a6c",
        markeredgecolor="#5d3a1a",
        linestyle="-",
        label="SAW Score",
    )
    ax.fill_between(x, y, alpha=0.2, color="#c49a6c")
    y_min, y_max = y.min(), y.max()
    y_range = y_max - y_min if y_max != y_min else 0.1
    y_padding = y_range * 0.1
    ax.set_ylim(bottom=y_min - y_padding, top=y_max + y_padding)
    ax.set_xlabel("Harga (USD)", fontsize=8, color="#4e342e")
    ax.set_ylabel("SAW Score", fontsize=8, color="#4e342e")
    ax.set_title(
        "Tren SAW Score terhadap Harga", fontsize=10, color="#4e342e",
fontweight="bold"
    )
    ax.set_facecolor("#faf0e6")
    fig.patch.set_facecolor("#f5e6d3")
    ax.grid(True, linestyle="--", alpha=0.3, color="#8d6e63")
    ax.legend(loc="best", fontsize=7)
    plt.tight_layout()
    return fig

```

```

# MAIN
def main():
    st.title("🕒 SmartWatchRank")
    st.markdown("### PEMILIHAN SMARTWATCH TERBAIK")
    st.markdown(
        "#### Menggunakan Metode Simple Additive Weighting (SAW) - SCPK 2025/2026"
    )
    st.markdown("---")

    df = load_data()
    if df.empty:
        st.stop()

    # Sidebar berisi semua kontrol: bobot, filter, tombol eksekusi
    with st.sidebar:
        st.image("smartwatch.png", width=250)
        st.markdown("## ⚙️ Bobot Kriteria")
        st.markdown("Total bobot akan otomatis = 1.00 (bobot relatif)")

        # Slider bobot
        raw_display = st.slider("📱 Ukuran Layar (Benefit)", 0.0, 1.0, 0.25, 0.01)
        raw_water = st.slider("💧 Ketahanan Air (Benefit)", 0.0, 1.0, 0.25, 0.01)
        raw_battery = st.slider("🔋 Baterai (Benefit)", 0.0, 1.0, 0.25, 0.01)
        raw_price = st.slider("💰 Harga (Cost)", 0.0, 1.0, 0.25, 0.01)

        # Normalisasi bobot agar total = 1
        total_raw = raw_display + raw_water + raw_battery + raw_price
        if total_raw == 0:
            total_raw = 1
        weights = {
            "Display Size": raw_display / total_raw,
            "Water Resistance": raw_water / total_raw,
            "Battery Life": raw_battery / total_raw,
            "Price": raw_price / total_raw,
        }

        st.markdown(f"***Total Bobot (setelah normalisasi):** 1.00")
        st.markdown(f"***Bobot akhir yang digunakan:**")
        st.markdown(f"- 📱 Ukuran Layar: {weights['Display Size']:.3f}")
        st.markdown(f"- 💧 Ketahanan Air: {weights['Water Resistance']:.3f}")
        st.markdown(f"- 🔋 Baterai: {weights['Battery Life']:.3f}")
        st.markdown(f"- 💰 Harga: {weights['Price']:.3f}")

        st.markdown(
            '<hr style="margin: 1rem 0; border: 1px solid #f5e6d3;">',
            unsafe_allow_html=True,
        )
        st.metric("📊 Jumlah Smartwatch", len(df))
        st.metric("🏷️ Jumlah Brand", df["Brand"].nunique())

    # Filter brand untuk menghitung hanya merek tertentu
    brand_filter = st.multiselect(

```

```

        "🔍 Filter Brand",
        options=sorted(df["Brand"].unique()),
        default=[],
        placeholder="Pilih brand...",
    )

    # Tombol Eksekusi
    if st.button("🧮 Hitung SAW", use_container_width=True,
type="primary"):
        # Ambil bobot dan filter brand dari state saat ini
        with st.spinner("Menghitung SAW..."):
            result_full, norm, details = process_saw(df, weights)
            if brand_filter:
                result_filtered = result_full[
                    result_full["Brand"].isin(brand_filter)
                ]
            else:
                result_filtered = result_full
            st.session_state.saw_result = result_filtered
            st.session_state.norm_df = norm
            st.session_state.norm_details = details
            st.success("Perhitungan selesai!")
        st.markdown(
            '<hr style="margin: 1rem 0; border: 1px solid #f5e6d3;">',
            unsafe_allow_html=True,
        )

    # Fitur menyimpan hasil dengan nama kustom
    calculation_name = st.text_input(
        "📝 Nama Perhitungan",
        value=f"Perhitungan {datetime.now().strftime('%H:%M:%S')}",
        key="calc_name_input",
    )

    if st.button("💾 Simpan Perhitungan ke Riwayat",
use_container_width=True):
        if st.session_state.saw_result is None or
st.session_state.saw_result.empty:
            st.warning(
                "Tidak ada hasil perhitungan. Klik 'Hitung SAW'
terlebih dahulu."
            )
        else:
            nama_untuk_disimpan = calculation_name.strip()
            if nama_untuk_disimpan == "":
                nama_untuk_disimpan = (
                    f"Perhitungan
{datetime.now().strftime('%H:%M:%S')}"
                )
            result = st.session_state.saw_result # ambil dari
session state
            timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            entry = {
                "id": len(st.session_state.history) + 1,
                "timestamp": timestamp,
                "name": nama_untuk_disimpan,
                "weights": weights.copy(), # bobot saat itu (bisa
disimpan)
                "top_watch": {

```

```

        "brand": result.iloc[0]["Brand"],
        "model": result.iloc[0]["Model"],
        "score": float(result.iloc[0]["SAW_Score"]),
    },
    "total_watches": len(result),
}
st.session_state.history.insert(0, entry)
st.session_state.saved_results[entry["id"]] =
result.copy()
save_history_to_csv(st.session_state.history) # Simpan
ke file permanen
save_saved_result(entry["id"], result)
st.success(f"✅ '{nama_untuk_disimpan}' tersimpan!")
st.rerun()

# TAB NAVIGASI
tab1, tab2, tab3, tab4, tab5 = st.tabs(
    [
        "📱 Data Smartwatch",
        "🏆 Hasil SAW",
        "📖 Riwayat Perhitungan",
        "📊 Visualisasi",
        "📄 Tentang",
    ]
)

# Tab Data Smartwatch
with tab1:
    st.markdown("## 📱 Data Smartwatch")
    st.markdown(
        "Berikut adalah data lengkap smartwatch yang tersedia dalam
sistem."
    )

    # Tampilkan dataframe tanpa filter (seluruh data)
    st.dataframe(df, use_container_width=True, height=500)

    # Opsi download sebagai CSV
    csv = df.to_csv(index=False).encode("utf-8")
    st.download_button(
        label="📄 Download Data sebagai CSV",
        data=csv,
        file_name="smartwatch_data.csv",
        mime="text/csv",
        use_container_width=True,
    )

# Tab Hasil SAW
with tab2:
    result_filtered = st.session_state.saw_result
    norm_df = st.session_state.norm_df
    norm_details = st.session_state.norm_details

    if result_filtered is not None and not result_filtered.empty:
        # Menampilkan ringkasan metrik (Total, Harga rata-rata,
        Baterai rata-rata, Ketahanan air)
        col_met1, col_met2, col_met3, col_met4 = st.columns(4)
        with col_met1:
            st.markdown(

```

```

class="metric-value">{len(result_filtered)}</div><div
class="metric-label">Total Smartwatch</div></div>',
    unsafe_allow_html=True,
)
with col_met2:
    avg_price = result_filtered["Price (USD)"].mean()
    st.markdown(
        f'<div class="metric-card"><div
class="metric-value">${avg_price:.0f}</div><div
class="metric-label">Harga Rata-rata</div></div>',
        unsafe_allow_html=True,
    )
with col_met3:
    avg_bat = result_filtered["Battery Life (days)"].mean()
    st.markdown(
        f'<div class="metric-card"><div
class="metric-value">{avg_bat:.1f}
class="metric-label">Baterai Rata-rata</div></div>',
        unsafe_allow_html=True,
    )
with col_met4:
    avg_water = result_filtered["Water Resistance
(meters)"].mean()
    st.markdown(
        f'<div class="metric-card"><div
class="metric-value">{avg_water:.0f}
class="metric-label">Ketahanan Air</div></div>',
        unsafe_allow_html=True,
    )
winner = result_filtered.iloc[0]

st.markdown("<br>", unsafe_allow_html=True)
col_win1, col_win2 = st.columns([1, 2])
with col_win1:
    st.markdown(
        f"""
        <div style="background: linear-gradient(135deg, #c49a6c,
#8b5a2b); border-radius: 20px; padding: 20px; text-align: center; color:
white;">
            <h1 style="color: white;">🏆</h1>
            <h2 style="color: white;">#1 Pilihan Terbaik</h2>
            <p>SAW Score: {winner['SAW_Score']:.4f}</p>
        </div>
        """,
        unsafe_allow_html=True,
    )
with col_win2:
    st.markdown(
        f"""
        <div style="background-color: #fff8f0; border-radius:
15px; padding: 20px;">
            <h2>🏆 {winner['Brand']} {winner['Model']}</h2>
            <table style="width:100%">
                <tr><td>📱 Ukuran
Layar</td><td><b>{winner['Display Size (inches)']}</b></td></tr>
                <tr><td>💧 Ketahanan
Air</td><td><b>{winner['Water Resistance (meters)']}</b></td></tr>
            </table>
        </div>
        """
    )

```

```

<tr><td>🔋 Baterai</td><td><b>{winner['Battery
Life (days)']}</b></td></tr>
<tr><td>💰 Harga</td><td><b>${winner['Price
(USD)']}</b></td></tr>
<tr><td>🎯 SAW
Score</td><td><b>{winner['SAW_Score']:.4f}</b></td></tr>
<tr><td>📊
Rank</td><td><b>#{winner['Rank']}</b></td></tr>
</table>
</div>
"""
    unsafe_allow_html=True,
)
st.markdown("<br>", unsafe_allow_html=True)

# Tabel hasil perbandingan (diurutkan dari tertinggi ke
terendah)
st.markdown("### 📋 Hasil Perhitungan SAW Lengkap")
display_cols = [
    "Rank",
    "Brand",
    "Model",
    "Display Size (inches)",
    "Water Resistance (meters)",
    "Battery Life (days)",
    "Price (USD)",
    "SAW_Score",
]
display_df = result_filtered[display_cols].copy()
display_df["SAW_Score"] = display_df["SAW_Score"].round(4)
st.dataframe(display_df, use_container_width=True,
height=400)

# Detail normalisasi (proses perhitungan)
with st.expander("🔍 Detail Normalisasi SAW"):
    st.markdown("***Parameter Normalisasi***")
    for k, v in norm_details.items():
        st.markdown(f"- **{k}**: {v}")
    st.markdown("***Nilai Normalisasi (10 baris pertama):***")
    st.dataframe(norm_df.head(10), use_container_width=True)
else:
    st.info(
        "🔍 Belum ada hasil. Klik tombol 'Hitung SAW' di sidebar
        terlebih dahulu."
    )

# Tab Riwayat Perhitungan
with tab3:
    st.markdown("### 📜 Riwayat Perhitungan SAW")
    if not st.session_state.history:
        st.info("Belum ada perhitungan yang disimpan.")
    else:
        # Tombol hapus semua riwayat
        if st.button("🗑️ Hapus Semua Riwayat"):
            st.session_state.history = []
            st.session_state.saved_results = {}
            # Hapus file
            if os.path.exists(HISTORY_FILE):
                os.remove(HISTORY_FILE)

```

```

for f in os.listdir(RESULTS_DIR):
    os.remove(os.path.join(RESULTS_DIR, f))
st.rerun()
for hist in st.session_state.history:
    with st.container():
        col1, col2, col3 = st.columns([2, 2, 1])
        with col1:
            st.markdown(
                f"""
                <div class="history-card">
                <b>🏆 {hist['name']}</b><br>
                ⌚ {hist['timestamp']}<br>
                🏆 Pemenang: {hist['top_watch']['brand']}
                {hist['top_watch']['model']}<br>
                📊 Score:
                {hist['top_watch']['score']:.4f}<br>
                📱 Total: {hist['total_watches']}
                </div>
                """,
                unsafe_allow_html=True,
            )
        with col2:
            w = hist["weights"]
            st.markdown(
                f"""
                <div class="history-card">
                ⚙️ Bobot:<br>
                📱 Ukuran Layar: {w['Display Size']:.3f}<br>
                💧 Ketahanan Air: {w['Water
Resistance']:.3f}<br>
                🔋 Baterai: {w['Battery Life']:.3f}<br>
                💰 Harga: {w['Price']:.3f}
                </div>
                """,
                unsafe_allow_html=True,
            )
        with col3:
            # Tombol untuk melihat detail (visualisasi) hasil
            tersimpan
            if st.button("👁️ Detail",
key=f"view_{hist['id']}"):
                st.session_state.view_id = hist["id"]
            if st.button("🗑️ Hapus",
key=f"del_{hist['id']}"):
                idx = [
                    i
                    for i, h in
enumerate(st.session_state.history)
                    if h["id"] == hist["id"]
                ][0]
                st.session_state.history.pop(idx)
                if hist["id"] in
st.session_state.saved_results:
                    del
st.session_state.saved_results[hist["id"]]
                    # Hapus file
                    file_path = os.path.join(
                        RESULTS_DIR, f"result_{hist['id']}.csv"
                    )

```

```

        if os.path.exists(file_path):
            os.remove(file_path)
        save_history_to_csv(st.session_state.history)
        st.rerun()
        st.markdown("---")
        # Menampilkan visualisasi dari hasil yang dipilih
        if (
            "view_id" in st.session_state
            and st.session_state.view_id in
st.session_state.saved_results
        ):
            detail_data =
st.session_state.saved_results[st.session_state.view_id]
            st.markdown("### 🔍 Visualisasi dari Perhitungan
Terseleksi")
            st.markdown(
                f"**Nama:** {next((h['name'] for h in
st.session_state.history if h['id']==st.session_state.view_id), '')}"
            )

            # Pie chart
            fig_pie = plot_pie_gradient(detail_data)
            st.pyplot(fig_pie)
            # Bar chart
            fig_bar = plot_bar_gradient(detail_data)
            st.pyplot(fig_bar)
            # Line chart
            fig_line = plot_line_chart_harga_saw(detail_data)
            st.pyplot(fig_line)

            if st.button("❌ Tutup Detail Visualisasi"):
                del st.session_state.view_id
                st.rerun()

# Tab Visualisasi
with tab4:
    st.markdown("### 📊 Visualisasi Data Smartwatch")
    result_filtered = st.session_state.saw_result
    if result_filtered is not None and not result_filtered.empty:
        # Pie chart
        fig_pie = plot_pie_gradient(result_filtered)
        st.pyplot(fig_pie)
        # Bar chart
        fig_bar = plot_bar_gradient(result_filtered)
        st.pyplot(fig_bar)
        # Line chart
        fig_line = plot_line_chart_harga_saw(result_filtered)
        st.pyplot(fig_line)
    else:
        st.info("🔍 Belum ada hasil. Klik tombol 'Hitung SAW' di
sidebar.")

# Tab Profil Kelompok
with tab5:
    st.markdown("### 👥 Tentang Kelompok")
    st.markdown(
        "Aplikasi **SmartWatchRank** dikembangkan sebagai proyek
Sistem Pendukung Keputusan (SPK) menggunakan metode **Simple Additive
Weighting (SAW)**."
    )

```

```

    )

    st.markdown(
        """
        <style>
            /* Memilih elemen gambar yang dihasilkan oleh st.image di
dalam kolom */
            .stColumn img {
                width: 150px !important;
                height: 150px !important;
                object-fit: cover !important;
                border-radius: 50% !important;
                border: 2px solid #c49a6c !important;
                box-shadow: 0 4px 8px rgba(0,0,0,0.1) !important;
                display: block !important;
                margin-left: auto !important;
                margin-right: auto !important;
            }
        </style>
        """,
        unsafe_allow_html=True,
    )

    # kolom profil anggota
    col1, col2 = st.columns(2, gap="large")

    with col1:
        try:
            st.image("titik.jpeg", width=180, caption="Titik Gemini")
        except:
            st.image(
                "https://cdn-icons-png.flaticon.com/512/3135/3135715.png",
                width=180,
                caption="Titik Gemini",
            )
            st.markdown("***Nama:** Shientya Belva")
            st.markdown("***NIM:** 123240042")
            st.markdown("***Kontribusi:** Desain UI/UX, Analisis Data &
Visualisasi")

    with col2:
        try:
            st.image("buncis.jpeg", width=180, caption="Buncis
Bosok")
        except:
            st.image(
                "https://cdn-icons-png.flaticon.com/512/3135/3135715.png",
                width=180,
                caption="Buncis Bosok",
            )
            st.markdown("***Nama:** Bylbiss El Haqqie")
            st.markdown("***NIM:** 123240003")
            st.markdown("***Peran:** Implementasi SAW")

    st.markdown("---")
    st.markdown("## 📖 Metode Simple Additive Weighting (SAW)")

```

```

st.markdown("""
    **Simple Additive Weighting (SAW)** adalah salah satu metode
    paling populer dalam Sistem Pendukung Keputusan multi-kriteria.
    Metode ini bekerja dengan melakukan **normalisasi** setiap
    kriteria terhadap nilai maksimum (untuk benefit) atau minimum (untuk
    cost),
    kemudian mengalikannya dengan bobot yang telah ditentukan, dan
    menjumlahkan semua nilai untuk setiap alternatif.

    **Langkah-langkah SAW yang diterapkan dalam aplikasi ini:**

    1. **Menentukan kriteria**
    - Ukuran Layar (Benefit) - semakin besar semakin baik
      - Ketahanan Air (Benefit) - semakin tinggi rating kedalaman
        semakin baik
      - Daya Tahan Baterai (Benefit) - semakin lama semakin baik
      - Harga (Cost) - semakin murah semakin baik

    2. **Memberikan bobot preferensi**
    Pengguna dapat mengatur bobot melalui sidebar. Bobot akan
    otomatis dinormalisasi sehingga totalnya = 1.

    3. **Normalisasi matriks keputusan**
    - Kriteria *benefit* dinormalisasi dengan: `nilai /
    nilai_maksimum`
    - Kriteria *cost* dinormalisasi dengan: `nilai_minimum / nilai`

    4. **Menghitung skor akhir**
    Setiap alternatif (smartwatch) dihitung skornya dengan
    menjumlahkan hasil perkalian nilai normalisasi dengan bobot masing-masing
    kriteria.

    5. **Perankingan**
    Smartwatch dengan skor SAW tertinggi menjadi rekomendasi terbaik.

    **Keunggulan SAW** adalah sederhana, mudah dipahami, dan cocok
    untuk kasus pemilihan produk dengan banyak kriteria seperti smartwatch.

    ---
    ### 🎯 Tujuan Aplikasi
    Membantu pengguna dalam memilih smartwatch yang paling sesuai
    dengan preferensi mereka secara objektif dan transparan.

    **SCPK 2025/2026** - Sistem Pendukung Keputusan Cerdas.
    """)

if __name__ == "__main__":
    main()

```

SmartWatchRank

PEMILIHAN SMARTWATCH TERBAIK

Menggunakan Metode Simple Additive Weighting (SAW)

Data Smartwatch
Hasil SAW
Riwayat Perhitungan
Visualisasi
Tentang

Data Smartwatch

Berikut adalah data lengkap smartwatch yang tersedia dalam sistem.

	Brand	Model	Display Size (inches)	Resolution	Water Resistance (meters)	Battery Life (days)	Price (USD)
0	Apple	Watch Series 7	1.9	396 x 484	50	18	399
1	Samsung	Galaxy Watch 4	1.4	450 x 450	50	40	249
2	Garmin	Venu 2	1.3	416 x 416	50	11	399
3	Fitbit	Versa 3	1.58	336 x 336	30	6	229
4	Fossil	Gen 6	1.28	416 x 416	30	24	299
5	Huawei	Watch 3 Pro	1.43	486 x 486	50	14	499
6	TicWatch	E3	1.3	360 x 360	50	2	199
7	Oppo	Watch 2	1.75	372 x 430	50	4	199
8	Amazfit	GTR 3	1.39	454 x 454	50	12	199

- Ukuran Layar: 0.328
- Ketahanan Air: 0.252
- Baterai: 0.218
- Harga: 0.202

Jumlah Smartwatch: 371

Jumlah Brand: 40

Filter Brand: Casio, Amazfit

Hitung SAW

Nama Perhitungan: Pilihan jam1

Simpan Perhitungan ke Riwayat

Data Smartwatch
Hasil SAW
Riwayat Perhitungan
Visualisasi
Tentang

30

Total Smartwatch

\$267

Harga Rata-rata

13.0 d

Baterai Rata-rata

77 m

Ketahanan Air

1

#1 Pilihan Terbaik

SAW Score: 0.5282

Amazfit GTR 2e

Ukuran Layar	1.95"
Ketahanan Air	50.0m
Baterai	45.0 days
Harga	\$139.0
SAW Score	0.5282
Rank	#5

Hasil Perhitungan SAW Lengkap

	Rank	Brand	Model	Display Size (inches)	Water Resistance (meters)	Battery Life (days)	Price (USD)	SAW_Score
5	6	Amazfit	GTR 2e	1.65	50	45	139	0.5282
6	7	Amazfit	Bip S	1.28	50	40	69	0.5274
9	10	Casio	G-Shock GBD-H1.000	1.2	200	14	399	0.5066

- Ukuran Layar: 0.328
- Ketahanan Air: 0.252
- Baterai: 0.218
- Harga: 0.202

Jumlah Smartwatch: 371

Jumlah Brand: 40

Filter Brand: Casio, Amazfit

Hitung SAW

Nama Perhitungan: Pilihan jam1

Simpan Perhitungan ke Riwayat

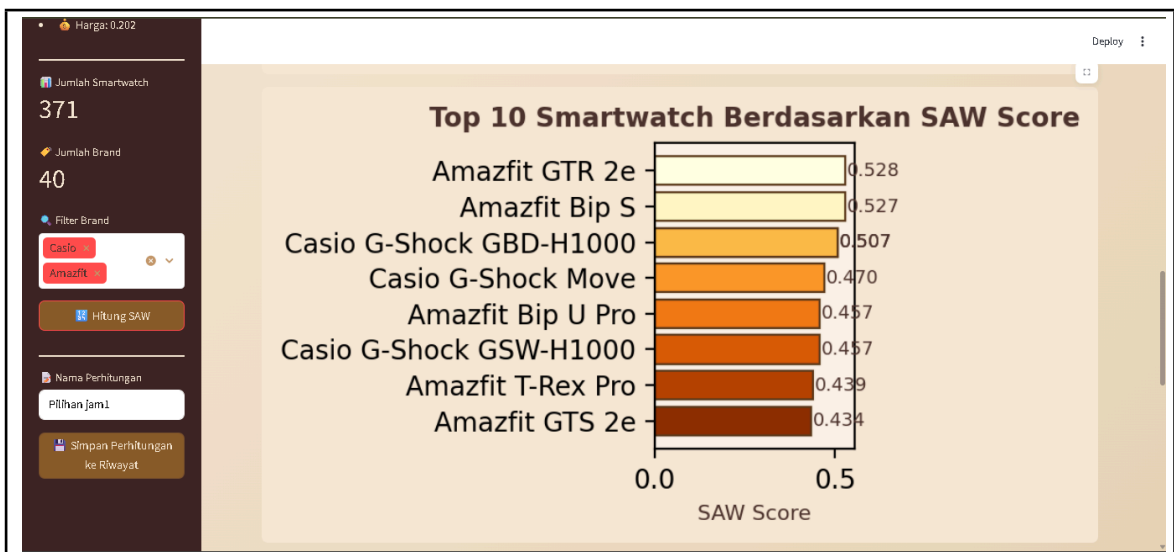
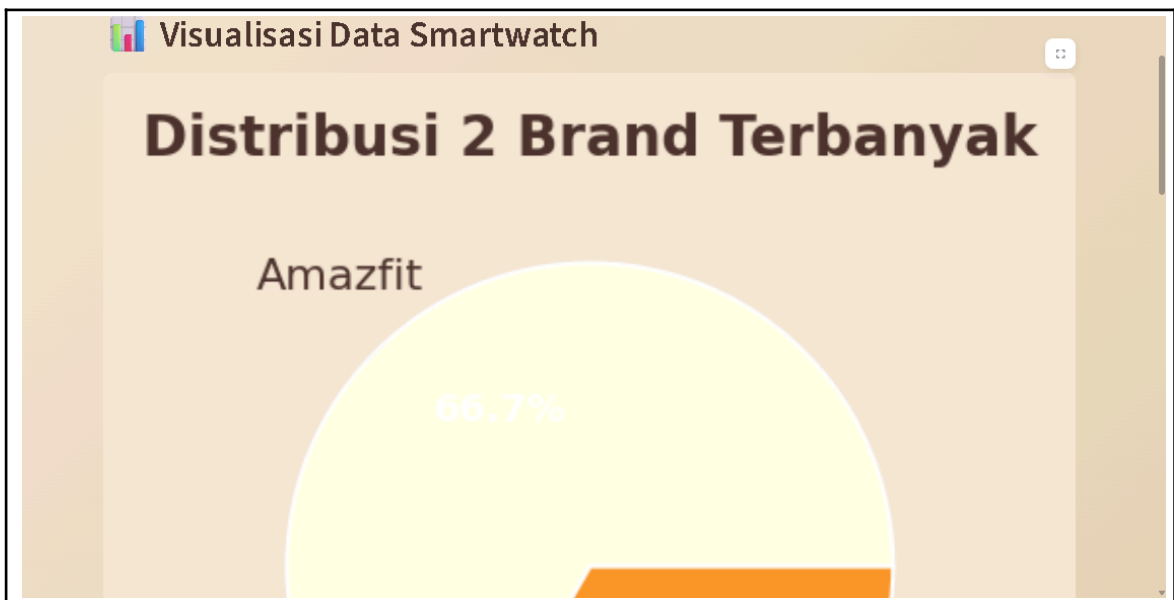
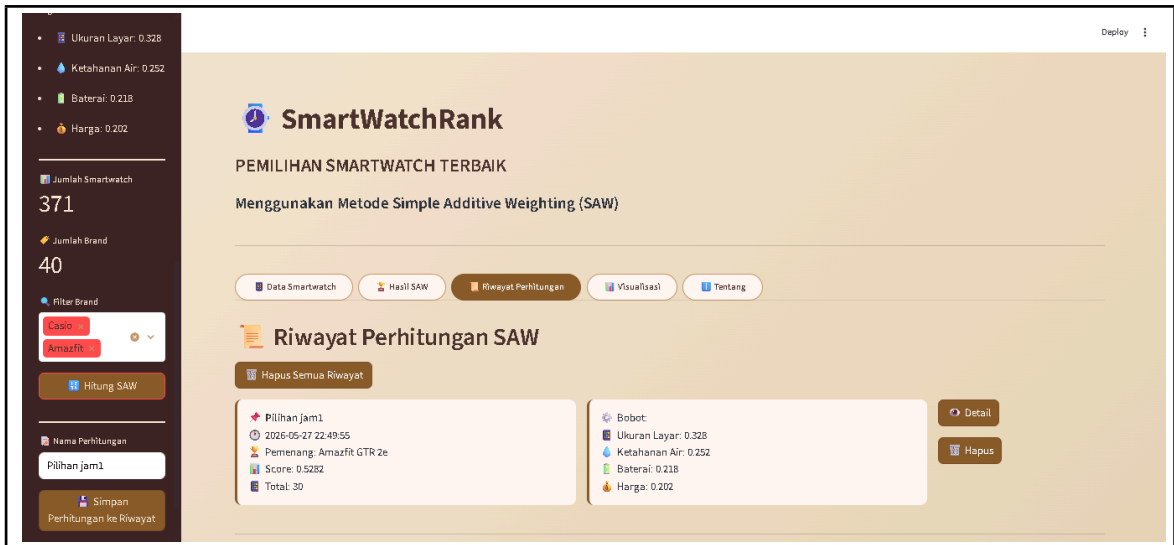
Detail Normalisasi SAW

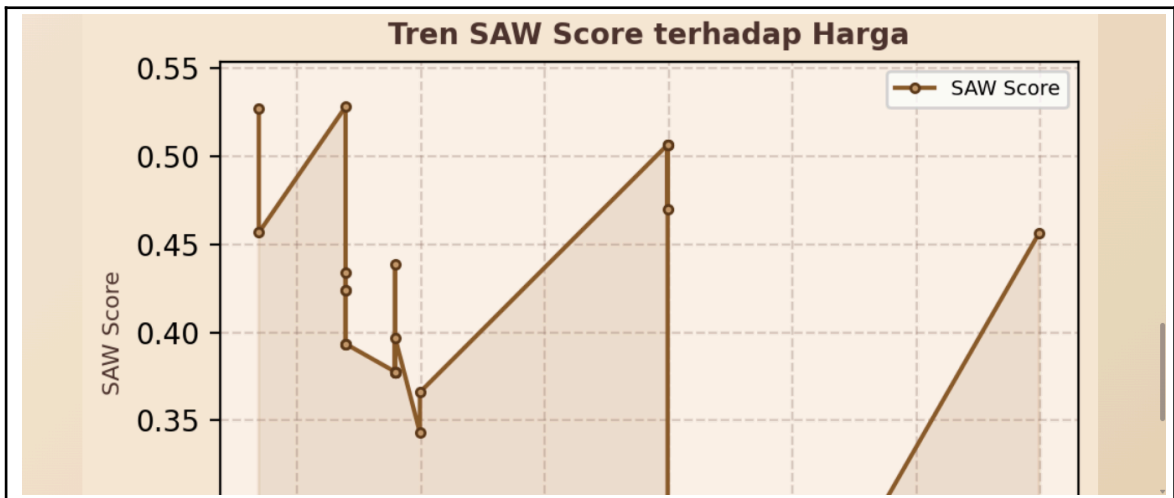
Parameter Normalisasi:

- Display Size: Benefit, max=2.10
- Water Resistance: Benefit, max=200
- Battery Life: Benefit, max=72.0
- Price: Cost, min=\$49

Nilai Normalisasi (10 baris pertama):

	Display Size	Water Resistance	Battery Life	Price	
0		0.9048	0.25	0.25	0.1228
1		0.8667	0.25	0.5556	0.1968
2		0.619	0.25	0.1528	0.1228
3		0.7524	0.25	0.0633	0.214
4		0.6095	0.15	0.3333	0.1639
5		0.681	0.25	0.1944	0.0982
6		0.619	0.25	0.0278	0.2452
7		0.8333	0.25	0.0556	0.2452
8		0.6519	0.25	0.1667	0.2452
10		0.7857	0.25	0.4167	0.1756





Data Smartwatch
Hasil SAW
Riwayat Perhitungan
Visualisasi
Tentang

👥 Tentang Kelompok

Aplikasi SmartWatchRank dikembangkan sebagai proyek Sistem Pendukung Keputusan (SPK) menggunakan metode Simple Additive Weighting (SAW).



Titik Gemini

Nama: Shientya Belva

NIM: 123240042

Kontribusi: Desain UI/UX, Analisis Data & Visualisasi



Bundis Bosok

Nama: Bylbiss El Haqqie

NIM: 123240003

Peran: Implementasi SAW

BAB III

JADWAL Pengerjaan dan Pembagian Tugas

3.1 Jadwal Pengerjaan

No	Kegiatan	2026				
		Minggu				
		1	2	3	4	dst
1	Mencari Dataset					
2	Membuat Konsep Proyek					
3	Mulai Membuat code Program					
4	Finalisasi Code Program dan Pembuatan Laporan					
dst						

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	Bylbiss El Haqqie	123240003	- Membantu Mencari Dataset - Membuat Code Program - Membuat Laporan
2	Shientya Belva	123240042	- Mencari Dataset - Membantu membuat Code Program - Membuat Laporan

BAB IV

KESIMPULAN DAN SARAN

4.2 Kesimpulan

Berdasarkan hasil perancangan dan implementasi proyek SmartWatchRank, dapat disimpulkan bahwa metode Simple Additive Weighting (SAW) berhasil diterapkan sebagai sistem pendukung keputusan untuk membantu pengguna dalam memilih smartwatch terbaik berdasarkan beberapa kriteria yang telah ditentukan. Sistem ini mampu melakukan proses evaluasi dan perankingan smartwatch secara objektif dengan mempertimbangkan empat kriteria utama, yaitu ukuran layar, ketahanan air, daya tahan baterai, dan harga.

Aplikasi yang dikembangkan menggunakan bahasa pemrograman Python dengan framework Streamlit sehingga menghasilkan antarmuka yang interaktif dan mudah digunakan. Pengguna dapat menentukan tingkat kepentingan setiap kriteria melalui pengaturan bobot yang kemudian dinormalisasi secara otomatis sebelum dilakukan perhitungan SAW. Hasil perhitungan ditampilkan dalam bentuk peringkat, nilai SAW, visualisasi data, serta rekomendasi smartwatch terbaik sesuai preferensi pengguna.

Selain itu, aplikasi juga menyediakan fitur pendukung seperti penyimpanan riwayat perhitungan, filter berdasarkan brand, tampilan detail normalisasi, serta berbagai grafik visualisasi yang membantu pengguna memahami hasil analisis dengan lebih mudah. Penggunaan dataset smartwatch dari Kaggle memungkinkan sistem melakukan evaluasi terhadap banyak alternatif sehingga hasil rekomendasi menjadi lebih representatif.

Dengan demikian, dapat disimpulkan bahwa aplikasi SmartWatchRank berhasil memenuhi tujuan proyek, yaitu membantu proses pengambilan keputusan dalam pemilihan smartwatch secara lebih cepat, objektif, transparan, dan berbasis data melalui penerapan metode Simple Additive Weighting (SAW). Metode SAW terbukti efektif untuk digunakan pada kasus pemilihan produk yang melibatkan banyak alternatif dan kriteria penilaian.

4.3 Saran

Beberapa saran untuk pengembangan sistem pada penelitian selanjutnya adalah:

1. Menambahkan lebih banyak kriteria penilaian, seperti fitur kesehatan, kualitas layar, kapasitas penyimpanan, dan kompatibilitas sistem operasi.
2. Menambahkan fitur pencarian dan filter yang lebih lengkap agar pengguna dapat menemukan smartwatch sesuai kebutuhan dengan lebih mudah.
3. Mengintegrasikan dataset yang selalu diperbarui sehingga informasi produk yang digunakan tetap relevan dan *up-to-date*.
4. Mengembangkan aplikasi ke versi mobile atau web yang dapat diakses secara lebih luas oleh pengguna.